

Academic Question Answering System Using Retrieval-Augmented Generation

Project Documentation · Project ID P_102

Aman Singh · September 2026

Repository: github.com/not-Lucifer/RAG-based-Question-Answering-System

1. Abstract

Students keep their study material as many PDFs (lecture notes, textbooks and previous-year papers), so finding one precise answer means searching every file by hand. General-purpose chatbots answer instantly but are not grounded in the student's syllabus, cannot say where an answer came from, and may be confidently wrong. This project is a **Retrieval-Augmented Generation (RAG)** system that answers questions **only from the student's own PDFs** and **cites the file and page** of every claim. Uploaded PDFs are extracted page by page, cleaned, split into overlapping chunks, embedded with a local sentence-transformer model and stored in a ChromaDB vector database. For each question the most relevant chunks are retrieved and passed to a large language model, a local LLaMA 3.2 3B served by Ollama or OpenAI's gpt-4o-mini, which answers with inline citations. When nothing relevant is found, the system replies *"I couldn't find this in your uploaded material."* instead of guessing.

The system supports follow-up questions, subject and document filters, three retrieval strategies (similarity, MMR and hybrid BM25 + vector), streaming answers and chat history, through a FastAPI backend and a Streamlit interface that run on an ordinary laptop. On a 30-question evaluation over three real course PDFs (53 pages), it retrieved the correct page in the top five for every answerable question (Hit@5 = 1.00, MRR = 0.95), refused all out-of-scope questions, and answered in about 5.4 s on a laptop GPU. All 102 automated tests pass offline.

2. Tech Stack

Layer	Technology	Purpose
Language	Python 3.13 (compatible with 3.11+)	Backend, pipeline, UI and scripts
Orchestration	LangChain 1.4 (LCEL)	Prompt templates, chains, retriever interfaces
PDF extraction	PyMuPDF 1.28	Page-wise text extraction with 1-based page numbers
Chunking	RecursiveCharacterTextSplitter	1000-character chunks, 150 overlap, never crossing pages
Embeddings	all-MiniLM-L6-v2 (local, CPU); OpenAI text-embedding-3-small (optional)	384-dimension sentence vectors
Vector store	ChromaDB 1.5	Persistent nearest-neighbour search with metadata filters
Keyword search	rank-bm25	Exact-term matching for hybrid retrieval
LLM	Ollama llama3.2:3b (local); OpenAI gpt-4o-mini (cloud)	Answer generation, switched by configuration
Backend API	FastAPI 0.142, Uvicorn, Pydantic v2	REST endpoints, validation, Server-Sent Events streaming
Persistence	SQLite + SQLAlchemy 2.1	Documents, chat sessions, messages, feedback
Frontend	Streamlit 1.64	Upload, chat, citations, filters, sessions
Configuration	pydantic-settings, .env, settings.yaml	Provider switch, tunables; secrets kept out of code

Layer	Technology	Purpose
Quality	pytest, FastAPI TestClient, Streamlit AppTest, ruff, black	Offline test suite, linting, formatting
Deployment	Render Blueprint (render.yaml)	API + UI web services with a persistent disk (prepared)

Key design decisions

- **Local-first:** embeddings and the LLM can run entirely on the laptop (MiniLM on CPU, LLaMA 3.2 3B on GPU); OpenAI is an optional configuration switch.
- **Refusal before generation:** if the best retrieved chunk scores below 0.25 cosine similarity, the system answers "not found" without building or calling the LLM; the grounded prompt is a second line of defence.
- **Index safety:** the embedding model's name is stored with the collection; a different model is refused (INDEX_MISMATCH) until the index is rebuilt, because vectors from different models are incomparable.
- **Security:** API keys live only in a git-ignored .env file, are masked when printed and redacted from logs; uploads are checked for size, type and PDF signature and stored under generated names.

3. Flow and Architecture

The system has two pipelines sharing one persistent vector store: an **ingestion pipeline** that runs when a document is uploaded, and a **query pipeline** that runs for every question.

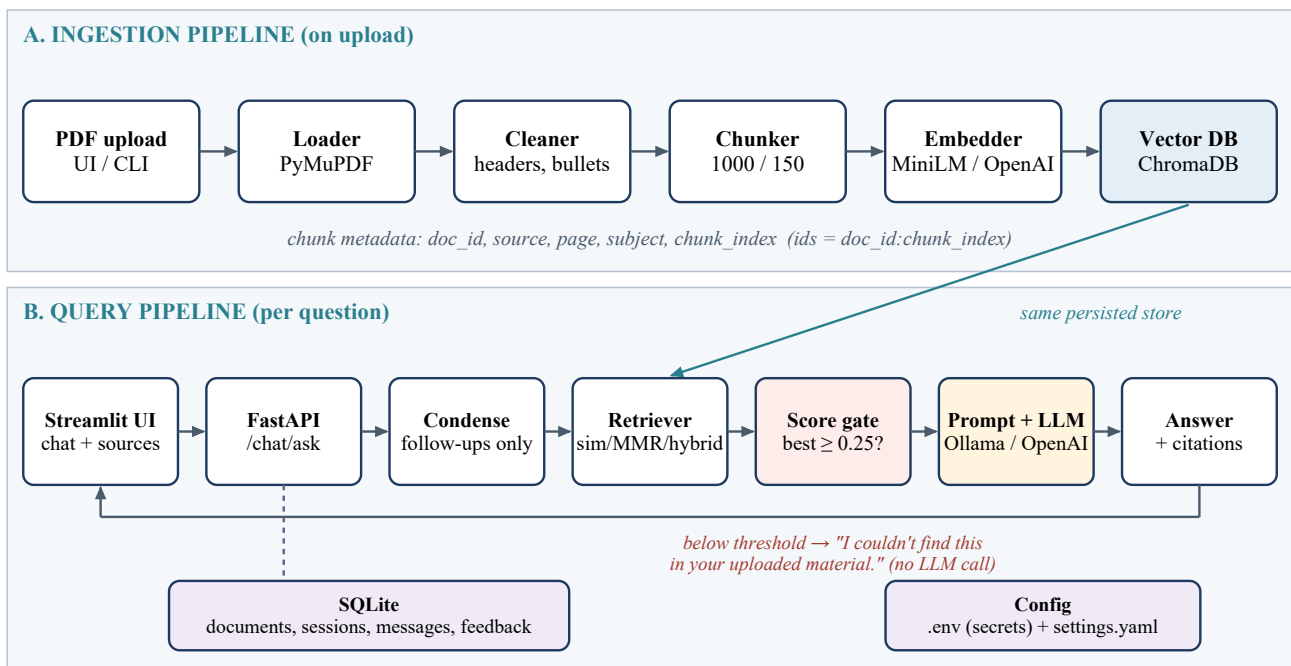


Figure 1. System architecture: ingestion and query pipelines.

Ingestion flow

The upload is validated (size limit, .pdf extension, MIME type, %PDF signature) and de-duplicated by SHA-256 hash, saved as <uuid>.pdf and registered in SQLite with status PROCESSING. PyMuPDF extracts one document per page; the cleaner removes running headers and footers (lines repeated on more than half the pages), bare page numbers and hyphenation breaks, re-joins text split one word per line and turns bullet glyphs into list items. Pages are split into overlapping chunks, embedded in batches of 32 and upserted into ChromaDB; the document becomes READY, or FAILED with a readable reason.

Query flow

- The UI sends the question, session id, filters and top-k to **POST /chat/ask** (or /chat/stream).
- The last four turns of the session are loaded; a follow-up such as "what are its types?" is rewritten into a standalone question. Complete questions are used as-is, and a rewrite that drops the question's topic is rejected.
- The retriever returns the top 5 chunks (similarity, MMR over 20 candidates, or hybrid BM25 + vector with weighted reciprocal rank fusion), each with a cosine relevance score; subject and document filters apply.
- Score gate: below 0.25 the system replies "I couldn't find this in your uploaded material." immediately.
- Otherwise the numbered context blocks [1] (file.pdf, p.12) and the question are sent to the LLM, whose system prompt allows answers only from that context, with inline citations.
- The question, answer, sources and latency are stored in SQLite; the UI shows the answer with expandable source cards (file, page, snippet) and feedback buttons.

Layered design

Layer	Code	Responsibility
Presentation	frontend/	Upload, chat, sources, settings; talks to the backend over HTTP only
API	app/api/routes/	Validation, routing, uniform {error, detail} error responses
Service	app/services/	Orchestrates ingestion and chat; owns database transactions
RAG core	app/ingestion, embeddings, vectorstore, retrieval, llm, chains	Pipeline logic; never imports the web framework, so it is testable in isolation
Data	ChromaDB, SQLite, data/raw	Vectors with metadata; documents, chats and feedback; original files

4. Directory Structure

Path	Contents
app/main.py	FastAPI application factory, CORS, error handlers, background model warm-up at startup
app/core/	config.py (settings from .env + settings.yaml), logging.py (secret redaction), exceptions.py (error codes)
app/api/routes/	health.py, documents.py (upload, list, delete, reindex), chat.py (ask, SSE stream), sessions.py (history, feedback)
app/schemas/	Pydantic request and response models
app/ingestion/	loader.py (PyMuPDF), cleaner.py, chunker.py, pipeline.py (load → clean → chunk → store)
app/embeddings/	factory.py: local MiniLM or OpenAI embeddings
app/vectorstore/	store.py: persistent ChromaDB, deterministic chunk ids, embedding-model check
app/retrieval/	retriever.py (similarity, MMR, filters, scores), hybrid.py (BM25 + RRF), reranker.py (optional)
app/llm/	factory.py (Ollama / OpenAI), prompts.py (system, QA and condense prompts)
app/chains/	rag_chain.py (LCEL chain, score gate, streaming), memory.py (history and follow-up rewriting)
app/db/, app/services/	SQLAlchemy engine, models and CRUD; document_service.py and chat_service.py
frontend/	streamlit_app.py, api_client.py, components/ (sidebar, chat_view, source_card)
scripts/	ingest_folder.py, ask_cli.py, reset_index.py, run_eval.py, make_samples.py
evaluation/	qa_dataset.json (30 questions), dataset.py (validation), metrics.py (Hit@k, MRR, LLM judges)

Path	Contents
tests/	9 test modules and conftest.py with fake embeddings and a fake chat model
config/settings.yaml	Chunking, retrieval, memory and embedding tunables
data/	samples/ (sample notes); raw/, vectorstore/ and app.db are created at run time and git-ignored
docs/	architecture.md, adr/ADR-001.md, report/, screenshots/
Project root	README.md, requirements.txt (+ -prod, -ui, .lock), .env.example, Makefile, make.ps1, pyproject.toml, render.yaml

5. Testing and Performance

Automated testing

The suite contains **102 tests** that run fully offline in about 30 seconds: a deterministic fake embedding model, a fake chat model and temporary directories mean no model downloads, no API calls and no changes to real data. Linting (ruff) and formatting (black) are clean.

Area	Tests	What is verified
Configuration, logging	7	Settings precedence, secret masking, API-key redaction in logs
Loader, cleaner, chunker	22	1-based pages, empty/invalid PDFs, header removal, fragment re-joining, bullets, overlap, metadata
Retrieval	13	All three modes, subject/document filters, BM25 cache invalidation, embedding-model mismatch
RAG chain and memory	33	Cited answers, refusal without an LLM call, follow-up rewriting safeguards, streaming
API	12	Upload → ask → follow-up → feedback → reindex → delete; every error code; path-traversal filenames
Frontend	8	Page rendering, streamed and non-streamed answers, backend down, interrupted answers
Evaluation code	7	Hit@k, MRR, judge-score parsing, dataset validation

Evaluation results

Evaluated on three uploaded course PDFs (magnetic circuits, DC/AC machines, communication skills; 53 pages) with 30 questions: 26 answerable, worded as an exam would ask them, and 4 that should be refused, including two near-domain traps (transformer EMF equation; stepper vs servo motor). Embeddings: all-MiniLM-L6-v2; answers and judging: llama3.2:3b on an RTX 4050 laptop GPU.

Metric	Result	Target
Hit@5 (correct page in top 5)	1.00 (all modes, chunk size 500 and 1000)	≥ 0.80
MRR (rank of first correct page)	0.95 (MMR, 1000); range 0.92–0.96	≥ 0.60
Refusal of out-of-scope questions	4 / 4 = 100% (threshold 1, LLM 3)	≥ 90%
Answer relevance (LLM judge, 1–5)	4.23	≥ 4.0
Faithfulness (LLM judge, 1–5)	3.46 – 3.69	≥ 4.0 (not met)
Latency per answer (local 3B model)	5.4 s	< 20 s

Performance

Situation	Time	Notes
First answer after starting the backend	5.6 s (was ~38 s)	Embedding model and LLM preloaded in the background at startup
Typical answer	~5 s	Generation-bound: ~68 tokens/s on the RTX 4050; reducing top-k 5 → 3 saved only 0.2 s
Follow-up question	+ ~3 s	One extra LLM call to rewrite the follow-up; skipped for complete questions
Out-of-scope question	~0.03 s	Refused by the score gate before any LLM call
Idle for 5+ minutes	No reload	Ollama keeps the model in GPU memory for 30 minutes (configurable)

Observations

- Refusal needs both layers: the similarity threshold alone stopped only 1 of 4 out-of-scope questions because the traps share vocabulary with the notes; the grounded prompt made the LLM refuse the remaining three.
- Faithfulness is below target and hard to measure with a 3B judge: between two runs 12 of 26 scores changed, some by 3 points, so differences under about ± 0.3 are noise. A stronger judge model would give steadier numbers.
- The weakest answers come from comparison tables, whose structure is lost in PDF text extraction; structure-aware table extraction and OCR for scanned notes are the main future improvements.